

ACKNOWLEDGEMENT

We wish to express our deep sense of gratitude to our project guide **Smt S.SHASHIKALA, ASSISTANT PROFESSOR** of Emerging Technologies in Computer Science Department, G. Pulla Reddy Engineering College, for his/her immaculate guidance, constant encouragement and cooperation which have made possible to bring out this project work.

We are grateful to our project in charge **Ms. S.SHABANA BEGUM, ASSISTANT PROFESSOR** of Emerging Technologies in Computer Science Department, G. Pulla Reddy Engineering College, for helping us and giving us the required information needed for our project work.

We are thankful to our Head of the Department **Dr. R. PRAVEEN SAM,** for his whole hearted support and encouragement during the project sessions.

We are grateful to our respected Principal **Dr. B.SREENIVASA REDDY,** for providing requisite facilities and helping us in providing such a good environment.

We wish to convey our acknowledgements to all the staff members of the Emerging Technologies in Computer Science department for giving the required information needed for our project work.

Finally, we wish to thank all our friends and well wishers who have helped us directly or indirectly during the course of this project work.

ABSTRACT

Title: End-to-end AI automation: From manual test cases to autonomous execution.

Domain: Computer Science - Machine Learning

Description: The automation of manual test cases, encompassing the generation, execution and maintenance processes, often poses challenges due to its time-consuming nature. Recognizing these challenges, we propose a comprehensive AI-driven solution designed to streamline the entire testing lifecycle. This solution aims to automate the conversion of manual test cases into automation code, covering API, UI, and Visual aspects of the application. Beyond just code generation, the tool extends its capabilities to handle the execution of test suites and automatic maintenance of test cases. An innovative feature of this tool is its ability to adapt to changes in manual test cases automatically, updating existing ones and incorporating new scenarios as needed. By providing automated report generation, the tool ensures that stakeholders receive timely insights into test results and performance metrics. This holistic approach is poised to save time, increase test coverage, and maintain up-to-date test suites. Ultimately, the AI-driven tool aspires to enhance software quality and development efficiency by introducing efficient and autonomous testing practices into the software development lifecycle.

CONTENTS

1. INTRODUCTION	Page No
1.1 Introduction	2
1.2 Motivation	4
1.3 Problem Definition	5
1.4 Objective of the Project	6
2. SYSTEM SPECIFICATIONS	
2.1 Software Specifications	8
2.2 Hardware Specifications	8
3. LITERATURE SURVEY	
3.1 Existing System	11
3.2 Disadvantages of Existing System	15
3.3 Proposed System	16
4. DESIGN	
4.1 Introduction	18
4.2 Flow chart of the Model	20
4.3 UML Diagrams	
4.3.1 Sequence Diagram	22
4.3.2 Use Case Diagram	23
4.3.3 Class Diagram	24

5. IMPLEMENTATION AND RESULTS

5.1 Introduction 26

5.2 Source Code 27

5.3 Output Code 42

6. TESTING AND VALIDATION 47

7. CONCLUSION AND FUTURE ENHANCEMENTS

7.1 Conclusion 50

7.2 Future Enhancements 50

8. REFERENCES 5

LIST OF FIGURES

FIGURE NO.	FIGURE NAME	PAGE NO.
4.1.1	Design Flow	19
4.2.1	Flow Chart	20
4.3.1	Sequence Diagram	22
4.3.2	Use Case Diagram	23
4.3.3	Class Diagram	24
5.3.1	AI Automation	42
	Tool Interface	
5.3.2	Automation Scripts	43
	Execution	
5.3.3	Reports Dashboard	44
5.3.4	Passed Test Case	44
5.3.5	Failed Test Case	45

INTRODUCTION

1. INTRODUCTION

In the world of software development, testing is a pivotal step to guarantee the functionality and reliability of the software. This process comes in two primary approaches: manual testing where individuals meticulously follow specific steps and automation testing where smart tools are used to accelerate and streamline the testing cycle. Beyond ensuring the software works well, testing is an essential component of the Software Development Life Cycle (SDLC). It aids in identifying and resolving bugs, errors, or glitches that might affect the software's performance. This iterative testing approach, integrated into the larger SDLC, ensures a comprehensive evaluation of the software at various stages, promoting quality and reducing the likelihood of issues reaching the end-users. The synergy of testing and the SDLC cycle is integral to delivering robust, bug-free software and maintaining a high standard of quality throughout the development process.

1.1 INTRODUCTION

Manual Testing: In the traditional landscape of software development, manual testing is the predominant method. Human testers meticulously follow predefined test cases, essentially acting as the quality assurance chefs in the kitchen. They execute test cases step by step, interacting with the software to identify defects and ensure that it meets specified requirements. This hands-on approach, akin to a chef crafting a dish, provides a thorough examination but can be time-consuming and susceptible to human error.

Challenges in Manual Testing: The manual testing process poses several challenges. Its labor-intensive nature makes it time-consuming, particularly for large-scale projects. Human error, influenced by factors like fatigue, can lead to inaccuracies. Additionally, repetitive tasks may result in monotony, impacting the precision and effectiveness of the testing process.

Automation Testing: With the rise of automation testing, a new culinary approach emerges. Automation involves the use of specialized software tools to execute test cases, reducing the manual effort required. It's like introducing an automated kitchen appliance to handle some of the cooking tasks. Automation allows for the swift execution of repetitive and large-scale test cases, significantly improving efficiency and test coverage.

Challenges Addressed by Automation: Automation resolves the challenges posed by manual testing. It accelerates the testing process, ensuring faster feedback on the software's performance. It minimizes human errors by executing test cases consistently and precisely. Additionally, automation enhances test coverage, allowing for the examination of a broader range of scenarios.

Challenges in Automation Testing: However, automation testing brings its own set of challenges. Developing and maintaining the specialized code for test automation can be complex and time-consuming. Automation is less adaptable to changes in the software, requiring continuous updates to the testing scripts. This can lead to challenges in keeping up with the evolving nature of software applications.

AI in Automation Testing: To address the above challenges, the report proposes embracing Artificial Intelligence (AI) within the automation testing process. AI, like a smart robot chef, can revolutionize testing by autonomously understanding and executing test cases. This involves leveraging AI models, such as OpenAI's ChatGPT, to automate the conversion of manual test cases into executable code. Just as a robot chef can follow a recipe on its own, AI automates the testing process, making it faster, smarter and more efficient.

In essence, integrating AI into automation testing represents a transformative leap, akin to introducing an intelligent sous-chef in the kitchen. It streamlines testing, adapts to changes effortlessly, and ensures a more intelligent and forward-looking approach to software quality assurance.

1.2 MOTIVATION

Enhancing Testing Efficiency: The project is motivated by a desire to enhance the efficiency of automation testing processes. The incorporation of AI aims to reduce manual effort and human errors, ultimately improving the overall effectiveness of testing procedures.

Optimizing Code Development: The motivation lies in optimizing the development and maintenance of automation code. By leveraging AI capabilities, the project seeks to streamline the conversion of manual test cases into automation scripts, addressing the complexities associated with code development in the testing landscape.

Improving Test Coverage: Motivated by the need for comprehensive testing, the project aims to automate the entire testing process. This includes the execution of a wide range of test cases, thereby enhancing test coverage and ensuring that potential issues are identified across various scenarios.

Addressing Technical Expertise Demands: Recognizing the technical expertise required for conventional automation testing, the project aims to democratize the testing process. By integrating AI, it seeks to make automation testing more accessible, reducing the barrier to entry and empowering a broader range of individuals to participate in testing activities.

Mitigating Human Errors: The project is motivated by the imperative to minimize human errors in testing. Automation, fueled by AI, is expected to provide a more reliable and accurate testing environment, reducing the likelihood of errors introduced by manual intervention.

Ensuring Adaptability to Evolving Software: The motivation extends to ensuring that the testing process remains adaptive to the rapid evolution of software. By incorporating AI, the project aims to create a testing framework that can autonomously adapt to changes in software functionality, ensuring its relevance over time.

Transforming Testing into an Intelligent Process: The overarching motivation is to transform the testing process into an intelligent and adaptive workflow. By leveraging AI, the project envisions a future where testing is not only automated but exhibits intelligent decision-making capabilities, aligning with the evolving demands of software development.

1.3 PROBLEM DEFINITION

The landscape of automation testing, while significantly enhancing efficiency, is marred by intricate challenges associated with the development and maintenance of the corresponding code. Despite the evident advantages of automation, creating and sustaining the specialized code demands a considerable investment of resources and poses complexities, creating a barrier to achieving a seamless and fully automated testing workflow.

One of the primary issues stems from the limitations inherent in current methods of automation testing, particularly when applied to website testing. These methods frequently exhibit a tendency to specialize either in User Interface (UI) or Application Programming Interface (API) testing, lacking a holistic and comprehensive approach to evaluating both aspects concurrently. This specialization restricts the ability to gain a complete understanding of how the UI and API components interact and affect each other, compromising the overall effectiveness of the testing process.

Manual intervention remains a significant requirement within many existing automation testing tools, limiting their autonomy. A case in point is Glider, a tool designed to extract UI scripts from websites using Artificial Intelligence (AI) models independently. However, there are worries about whether the tool's results can match what users expect and meet optimization criteria. The fact that the tool needs manual help takes away from its ability to work independently, affecting how fast and efficient the testing process can be. Furthermore, AI tools commonly employed in automation testing are often limited to creating one test case at a time, contributing to inefficiencies and a lack of scalability. This limitation impedes the ability to conduct comprehensive and simultaneous testing across various scenarios, reducing the overall thoroughness and coverage of the testing process.

In response to these challenges, this report emphasizes the critical need for a transformative solution. The proposed remedy involves the strategic incorporation of Artificial Intelligence (AI) into the automation testing process. The overarching goal is to comprehensively address the complexities associated with developing and maintaining automation testing code, ultimately achieving a fully automated testing workflow. By leveraging AI, the intention is to enhance adaptability, efficiency, and autonomy in the testing process, ensuring a more robust and streamlined approach to software quality assurance in the ever-evolving landscape of technology.

1.4 OBJECTIVE OF THE PROJECT

The primary objective of this transformative initiative is to harness the formidable capabilities of Artificial Intelligence (AI) in revolutionizing the automation testing process. In a rapidly evolving landscape of software development, the project aspires to introduce a paradigm shift by incorporating sophisticated Large Language Models (LLMs), such as OpenAI's GPT. The focal point lies in not only automating the test procedures but, more profoundly, in optimizing and enhancing the entire testing lifecycle.

At its core, the project seeks to address the intricate challenges associated with traditional automation testing methodologies. It recognizes that while automation has been a staple in testing for some time, the process of developing and maintaining the associated code remains intricate and resource-intensive. The aim is not merely efficiency through automation but a comprehensive transformation of the testing workflow.

By integrating LLMs into the testing framework, the project strives to impart a level of intelligence and adaptability that goes beyond conventional automated testing. The emphasis is on streamlining the conversion of manual test cases into automation code through the understanding and interpretation of natural language. This not only makes the process more intuitive but also significantly reduces the dependency on manual intervention, ultimately mitigating the risk of human errors.

The envisioned outcome extends beyond the realm of mere automation. It envisions a testing process that is not only seamless but also intelligent, capable of understanding and responding to changes in software requirements dynamically. The overarching goal is to empower the testing framework to evolve alongside the software it evaluates.

In summary, the project's objective is not just about automating the automation testing process; it is about creating an intelligent, adaptive, and forward-looking testing framework. The infusion of AI, particularly LLMs, aims to redefine how we approach testing, fostering an environment that is not only efficient but also inherently intelligent, ultimately contributing to the enhancement of overall software quality.

SYSTEM SPECIFICATIONS

2. SYSTEM SPECIFICATIONS

A robust combination of hardware and software are required to fulfill advanced testing needs. The specified requirements ensure optimal performance and compatibility with the sophisticated tools and frameworks involved in the testing environment.

2.1 SOFTWARE SPECIFICATIONS

The software requirements for the "End-to-End AI Automation: From Manual Test Cases to Autonomous Execution" project play a pivotal role in shaping a coherent and efficient testing environment. Operating within a Windows environment ensures compatibility with a wide array of testing tools and frameworks. The selected testing tools, including Selenium, Rest Assured, TestNG, Maven, and Extent Reports, form a powerful suite that collectively supports various aspects of automated testing, from web application testing to API testing and reporting. Leveraging Java as the primary programming language provides the versatility and robustness required for implementing intricate test scripts and interacting with diverse testing frameworks. The specified web browsers, Chrome, Edge, and Firefox, cover a broad spectrum of browser compatibility scenarios, ensuring comprehensive test coverage. The IntelliJ Idea Integrated Development Environment (IDE) offers a feature-rich platform for test script development, debugging, and overall project management. Additionally, the integration of OpenAI's GPT, a Large Language Model (LLM), brings a cutting-edge layer of AI-driven capabilities to the project, enabling natural language processing and enhancing the automation of manual test case conversion. These software specifications collectively contribute to creating a cohesive and adaptable testing ecosystem, emphasizing compatibility, efficiency, and advanced AI integration for a comprehensive testing approach.

2.2 HARDWARE SPECIFICATIONS

The hardware requirements for the "End-to-End AI Automation: From Manual Test Cases to Autonomous Execution" project are strategically chosen to provide a robust foundation for advanced testing capabilities. With 16 GB or more of RAM, the system gains the ability to efficiently handle multiple tasks concurrently, crucial for executing complex test scripts and managing large datasets in AI-driven testing scenarios. An Intel Quad Core processor or higher acts as the processing powerhouse, ensuring swift and efficient execution

of test instructions, particularly in scenarios involving parallel test executions. The substantial storage capacity of 500 GB or more is indispensable for retaining extensive datasets, test scripts, and the testing environment, facilitating traceability, analysis, and future reference. A stable and fast internet connection is essential for seamless integration of web-based testing tools, accessing external resources, and maintaining an up-to-date testing environment. The reliability of the system is paramount, minimizing the risk of unexpected failures and ensuring consistent and dependable test outcomes. The ability for parallel execution, made possible by higher RAM and processor capacity, significantly reduces test execution time by running multiple test cases simultaneously, whether in different browsers or on different threads. These specifications not only address the current testing needs but also future-proof the testing environment, providing scalability to handle increased test complexity and evolving testing requirements. Overall, the chosen hardware requirements form a comprehensive and strategic foundation to support the dynamic and resource-intensive nature of modern testing practices, integrating AI-driven testing and automation seamlessly

SOFTWARE REQUIREMENTS

- Operating System : Windows
- Testing Tools : Selenium, Rest Assured, TestNG, Maven, Extent Reports
- Programming Language : Java
- Browsers : Chrome, Edge, Firefox
- IDE : IntelliJ Idea
- LLM : OpenAI GPT

HARDWARE REQUIREMENTS

- RAM : 16 GB or more
- Processor : Intel Quad Core or higher
- Storage : 500 GB or more
- Internet Connectivity : Stable and Fast

LITERATURE SURVEY

3. LITERATURE SURVEY

3.1 EXISTING SYSTEM

3.1.1 Automation Testing Tools: A Comparative View

Authors: Dilara Ateşoğulları, Alok Mishra

Published Year: 2020

In the world of software development, ensuring the quality and efficiency of software through rigorous testing is paramount. The research paper titled "AUTOMATION TESTING TOOLS: A COMPARATIVE VIEW" serves as a cornerstone in the literature on software testing, providing an exhaustive comparison of twenty-one automation testing tools. This comparison is grounded on twenty distinctive attributes, highlighting the evolving landscape of automation testing in the software development process.

The paper begins by establishing the critical role of automation testing tools in enhancing the software testing phase, asserting that these tools significantly expedite the testing process, thereby facilitating timely software completion and implementation. It addresses a crucial dilemma faced by practitioners and researchers alike: the selection of an appropriate automation testing tool and framework. The objective is to furnish a comprehensive assessment that aids in navigating the plethora of available tools, ensuring that stakeholders are equipped with the knowledge to make informed decisions tailored to their specific testing needs.

The authors meticulously delineate the advantages of automation testing over traditional manual testing methods. Among the benefits cited are higher testing efficiency, improved accuracy and reliability, and the reusability and repeatability of test scripts. Additionally, automation testing is lauded for its capacity to simulate user environments, achieve greater test coverage, and detect bugs early in the development cycle, which, in turn, saves time and reduces costs.

A pivotal section of the paper is devoted to discussing the application domains of automation tests, emphasizing their suitability for regression tests, data-driven tests, smoke tests, and load and performance tests, among others. This delineation is crucial, as it

underscores the versatility of automation testing tools in addressing both functional and non-functional aspects of software quality.

Moreover, the study provides an insightful historical perspective on the adoption of test automation, citing statistics from the International Software Testing Qualifications Board that illustrate a significant increase in the utilization of automation testing tools over recent years. This trend underscores the growing recognition of the value that automation brings to the software testing process.

The comparative analysis presented in the paper is both extensive and nuanced, covering a wide array of tools, including widely recognized names such as Selenium, Unified Functional Testing (UFT), and TestComplete, as well as emerging tools that are shaping the future of automation testing. Each tool is evaluated against a comprehensive set of criteria, offering readers a clear understanding of their capabilities, strengths, and potential limitations.

In summary, the paper provides a valuable resource for both practitioners in the field of software testing and academic researchers. By offering a detailed comparison of a broad spectrum of automation testing tools, the paper contributes significantly to the body of knowledge in the field, guiding stakeholders towards making well-informed choices that align with their specific testing requirements and objectives.

3.1.2 Analysis and Design of Selenium WebDriver Automation Testing Framework

Authors: Satish Gojare, Rahul Joshi and Dhanashree Gaigaware

Published Year: 2015

The research paper presents a meticulous analysis and design blueprint for an automation testing framework utilizing Selenium WebDriver, with a particular emphasis on elevating the efficiency and robustness of web application testing. The framework's architectural intricacies are comprehensively explored, emphasizing its seamless integration with Selenium WebDriver and TestNG. This integration not only facilitates the creation, management, and execution of test cases but also significantly enhances accuracy while reducing manual effort.

A critical facet of the framework under scrutiny is its sophisticated architecture, featuring key components strategically designed to optimize testing processes. The document delves into the object repository, a pivotal element for managing web elements effectively. Furthermore, utility functions tailored for common tasks are scrutinized, contributing to the overall efficiency of the testing framework. The incorporation of data-driven testing methodologies is also thoroughly examined, emphasizing its role in fostering scalability and maintainability.

The discourse extends to discussing the advantages intrinsic to utilizing Selenium WebDriver for automated testing purposes. Notably, the framework capitalizes on Selenium's support for multiple browsers and operating systems, offering a versatile testing environment. Additionally, Selenium's ability to replicate user interactions with web applications is underscored, providing a more natural and accurate simulation.

Addressing the challenges encountered during the framework's developmental phase, the document sheds light on issues related to web element locators and dynamic content management. Solutions to overcome these obstacles are presented, showcasing the research's commitment to providing practical insights into problem-solving within the automation testing domain.

To substantiate the framework's efficacy, the document incorporates case studies and empirical data. The research demonstrates how the proposed framework significantly reduces test cycle times, enhances test coverage, and improves the overall maintainability of test scripts. The conclusion encapsulates the framework's profound contribution to the broader field of software testing, emphasizing its potential to streamline testing processes, elevate product quality, and facilitate the seamless integration of continuous integration and deployment practices.

3.1.3 The Impact of Artificial Intelligence on Software Testing

Authors: Hussam Hourani, Ahmad Hammad, Mohammad Lafi

Published Year: 2019

This paper delves into the profound impact of Artificial Intelligence (AI) on our daily lives, permeating various applications and systems that have become integral to our surroundings. With an ever-increasing deluge of data from diverse sources, the need for effective monitoring, analysis, and actionable insights has become paramount. In the intricate landscape of modern software applications, where timely releases compliant with Business Requirements are critical, AI emerges as a crucial force in the realm of Software Testing.

The paper meticulously explores the key pillars of AI that can be leveraged to enhance Software Testing practices. It sheds light on how these AI-driven advancements not only optimize testing processes but also lay the groundwork for the future of AI in the field. The discussion centers around the potential of AI to deliver more precise and efficient results, addressing the growing demands for timely and accurate testing.

As the findings suggest, AI-driven testing is poised to usher in a new era of quality assurance (QA) work. The capabilities of AI are showcased in achieving superior outcomes, promising a paradigm shift in how testing is approached. The paper anticipates significant benefits, including a reduction in time to market, increased organizational efficiency in producing sophisticated software, and the introduction of more intelligent automated testing solutions.

This paradigm shift towards AI-driven Software Testing is not merely a technological evolution; it is a transformative force that is set to reshape the landscape of quality assurance, promising a future where AI takes center stage in ensuring the robustness and efficiency of software testing processes.

3.2 DISADVANTAGES OF EXISTING SYSTEMS

While traditional automation testing tools and AI-driven automation testing tools have demonstrated success in various testing tasks, each approach comes with its own set of disadvantages and challenges.

Test Scenario Generation: Crafting comprehensive test scenarios manually can be time-consuming. AI-driven automation has the potential to intelligently generate test scenarios based on system understanding, reducing the manual effort in test case creation.

High Initial Investment: Implementing automation testing often incurs a significant upfront cost. The initial investment includes expenses related to tool acquisition, training, and infrastructure setup, which can be substantial for organizations.

High Skill Level Requirement: Implementing automation may demand a high level of technical expertise. Test developers require specialized skills of automation frameworks and tools to effectively contribute to and maintain the system.

Limited Self-Healing Mechanisms: Current automation tools lack advanced self-healing capabilities. When an element is not found or a script fails, it often requires manual intervention to identify and fix the issue. AI-driven automation aims to introduce self-healing mechanisms that can adapt to changes in the application.

Data-Driven Testing Complexity: Performing data-driven testing with traditional tools can be complex and may involve the creation of extensive datasets. AI-driven automation can streamline data-driven testing by intelligently generating and managing test data.

Cross-Browser Compatibility Challenges: Ensuring cross-browser compatibility is a common challenge in automation. Test scripts need to be crafted for each browser, and maintaining compatibility across different browsers requires additional effort. AI-driven tools can optimize cross-browser testing processes.

Maintenance Challenges: Maintenance of automation scripts poses a considerable challenge, particularly when managing extensive test suites. When the software undergoes frequent changes, updating manual test cases becomes imperative. Until the automation scripts are synchronized with these modifications, the test cases will consistently fail, underscoring the significance of timely updates to ensure alignment with the evolving software landscape.

Learning Curve: Conventional automation tools often have a steep learning curve for beginners.

3.3 PROPOSED SYSTEM

The proposed solution introduces a "low code" approach to automation testing, this innovative approach is meticulously crafted to cater to the testing needs of diverse websites and applications. It streamlines the automation process by leveraging Large Language Models (LLMs) to effortlessly translate manual UI and API test cases into executable automation scripts. This transformative system eliminates the need for intricate code development, allowing testers to focus on defining test scenarios without delving into complex scripting languages.

Once the manual test cases are converted into automation scripts, the system operates autonomously, executing these scripts with precision. The autonomous execution is complemented by a robust reporting mechanism that provides detailed insights into the success or failure of each test case. Additionally, the reports comprehensively capture any issues encountered during the testing process, facilitating efficient debugging and issue resolution.

By incorporating LLMs into the automation testing workflow, the solution goes beyond conventional methods, significantly reducing the risk of human errors in the scripting phase. This not only expedites the testing lifecycle but also enhances the reliability of the testing outcomes. The ultimate objective is to present a holistic and automated resolution to the prevalent challenges in automation testing.

This approach embodies a strategic fusion of cutting-edge language models with the practical needs of automation testing, ensuring a seamless and efficient testing procedure. The project's vision is to provide testers with a powerful tool that not only simplifies the automation process but also addresses the intricacies and challenges associated with scripting, thereby revolutionizing the landscape of automation testing.

DESIGN

4. DESIGN

4.1 INTRODUCTION

The design of our project is intricately crafted to revolutionize the landscape of software testing. At its core, the design amalgamates proven testing tools like Selenium, Rest Assured, and TestNG with cutting-edge technologies such as OpenAI's GPT. This fusion aims to offer a comprehensive testing solution that transcends traditional boundaries. By intricately weaving together these tools, our design sets the stage for a seamless, intelligent, and automated testing process.

The workflow involves a seamless interaction with OpenAI GPT. Test guidelines and manual test cases are passed to the OpenAI API, which, in turn, returns the generated code for UI and API automation scripts. These scripts are then executed using Selenium and Rest Assured, and the outcomes are meticulously captured and presented in Extent Reports. This innovative workflow not only ensures accuracy and reliability but also significantly expedites the test script creation process, marking a groundbreaking shift in the traditional approach to automation testing. Let's delve into the details of each component and understand how they synergistically contribute to the project's overarching goal.

Selenium: Selenium serves as the cornerstone for executing UI tests on web applications. It is a robust open-source framework that provides a suite of tools for automating web browsers. By utilizing Selenium, the project ensures the efficient and accurate execution of UI test cases across various web applications.

Rest Assured: Rest Assured is a powerful Java library designed specifically for automating RESTful API testing. In our project, it plays a crucial role in executing API tests, allowing for the validation of web services, endpoints, and data flow. This ensures comprehensive test coverage for both UI and API aspects of the applications under evaluation.

TestNG: TestNG, short for Test Next Generation, is a testing framework inspired by JUnit and NUnit. It facilitates parallel execution, data-driven testing, and the creation of robust test suites. TestNG is employed in our project to structure and manage test cases effectively, enhancing the scalability and maintainability of the testing framework.

Maven: Maven is a powerful project management tool that comprehensively handles the project's build lifecycle. It manages dependencies, compiles source code, runs tests, and packages the application. In our design, Maven streamlines the build process, ensuring consistency and efficiency throughout the development and testing phases.

Extent Reports: Extent Reports is an HTML reporting library widely used for generating interactive and detailed test reports. It provides insights into test execution results, making it an invaluable tool for tracking the success and failure of test cases. Extent Reports is integrated into the project to create informative and visually appealing reports for better analysis and debugging.

OpenAI GPT: OpenAI GPT, or Generative Pre-trained Transformer, represents the pinnacle of language models in our project. It transforms the manual test cases and guidelines provided into executable code for both UI and API automation scripts. By leveraging the power of AI, OpenAI GPT ensures accuracy, reduces human error, and accelerates the test script creation process.

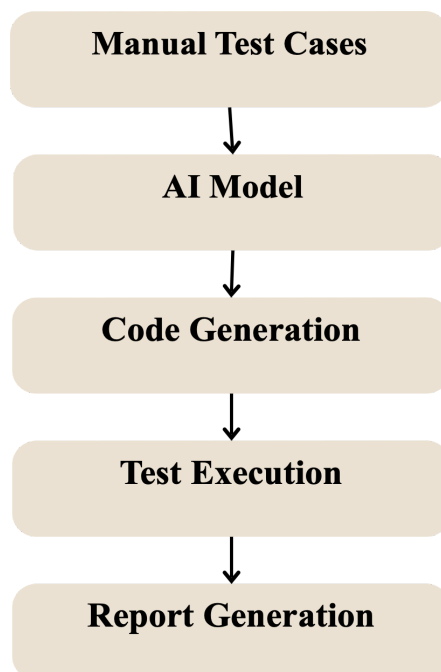


Fig- 4.1.1: Design Flow

4.2 FLOW CHART OF THE MODEL

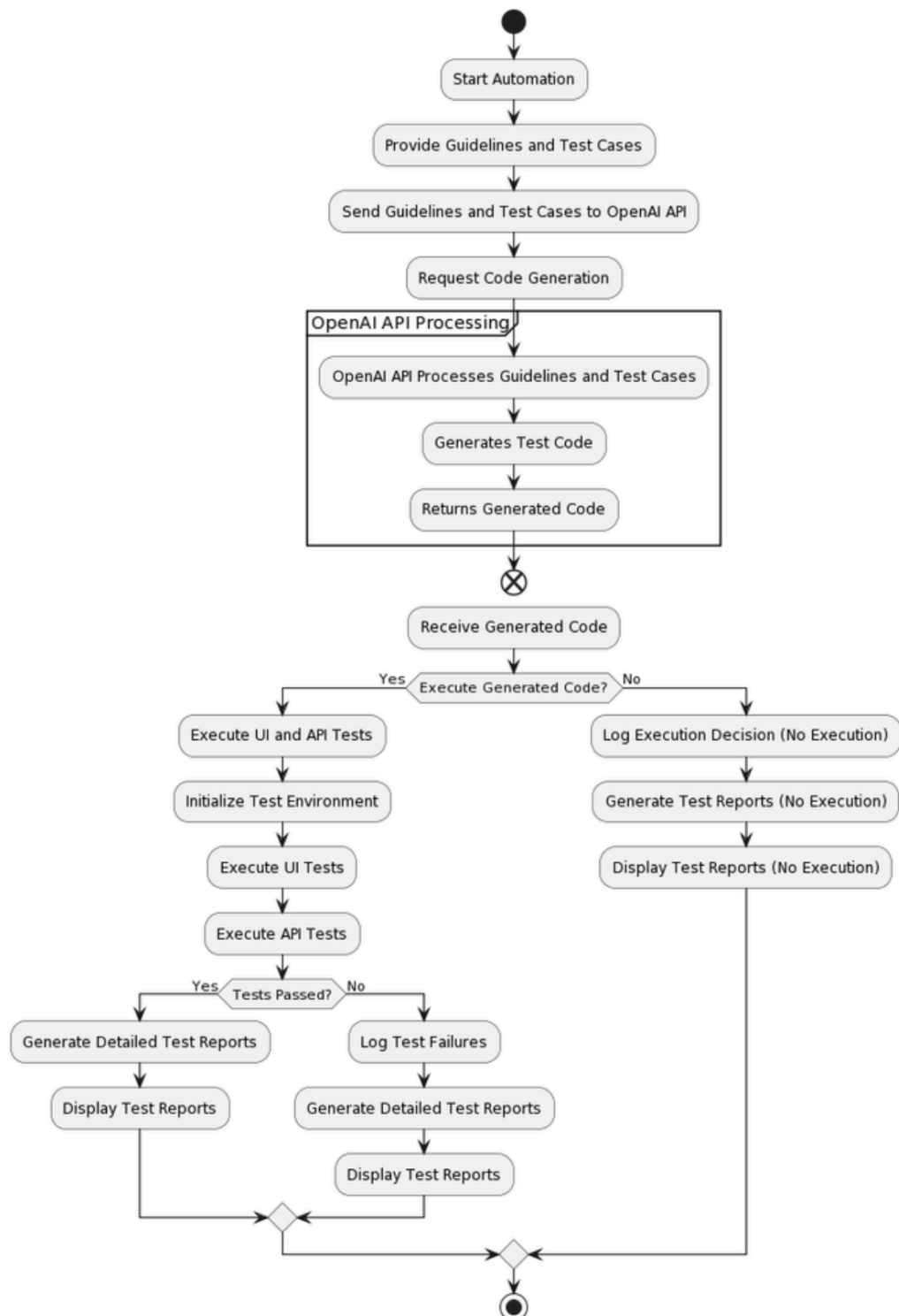


Fig- 4.2.1: Flow Chart

Start Automation: This initiates the entire workflow for automating test cases using AI.

Provide Guidelines and Manual Test Cases: In this step, AI is provided with coding guidelines and manual test cases which could include various scenarios and expected outcomes that the AI should handle.

Send Guidelines, Test Cases to OpenAI API and Request Code Generation: Guidelines and Test Cases are transmitted to the OpenAI API, indicating a request for the AI to process this information and generate test code based on the provided guidelines and test cases.

OpenAI API Processing:

Processes Guidelines and Test Cases: The OpenAI LLM interprets the provided guidelines and manual test cases, understanding the requirements and the testing context.

Generates Test Code: Leveraging the LLM capabilities, the AI generates test code that would fulfill the test cases. This could involve scripting for various testing frameworks or environments.

Returns Generated Code: The API then returns the generated test scripts/code to the user.

Receive and Execute Generated Code: The test scripts are received and the system proceeds to run the generated test scripts. This involves initializing the test environment, executing the tests (which could be UI, API, or unit tests), and then evaluating whether the tests pass or fail.

Generate Detailed Test Reports: Regardless of the outcome, detailed reports are generated to document the findings from the test execution or the hypothetical outcome based on the AI-generated code.

Display Test Reports: These reports are then presented to the user, providing a comprehensive view of the testing process, the results obtained, and any insights gleaned from the automation.

4.3 UML DIAGRAMS

4.3.1 SEQUENCE DIAGRAM - WORKING OF SELENIUM

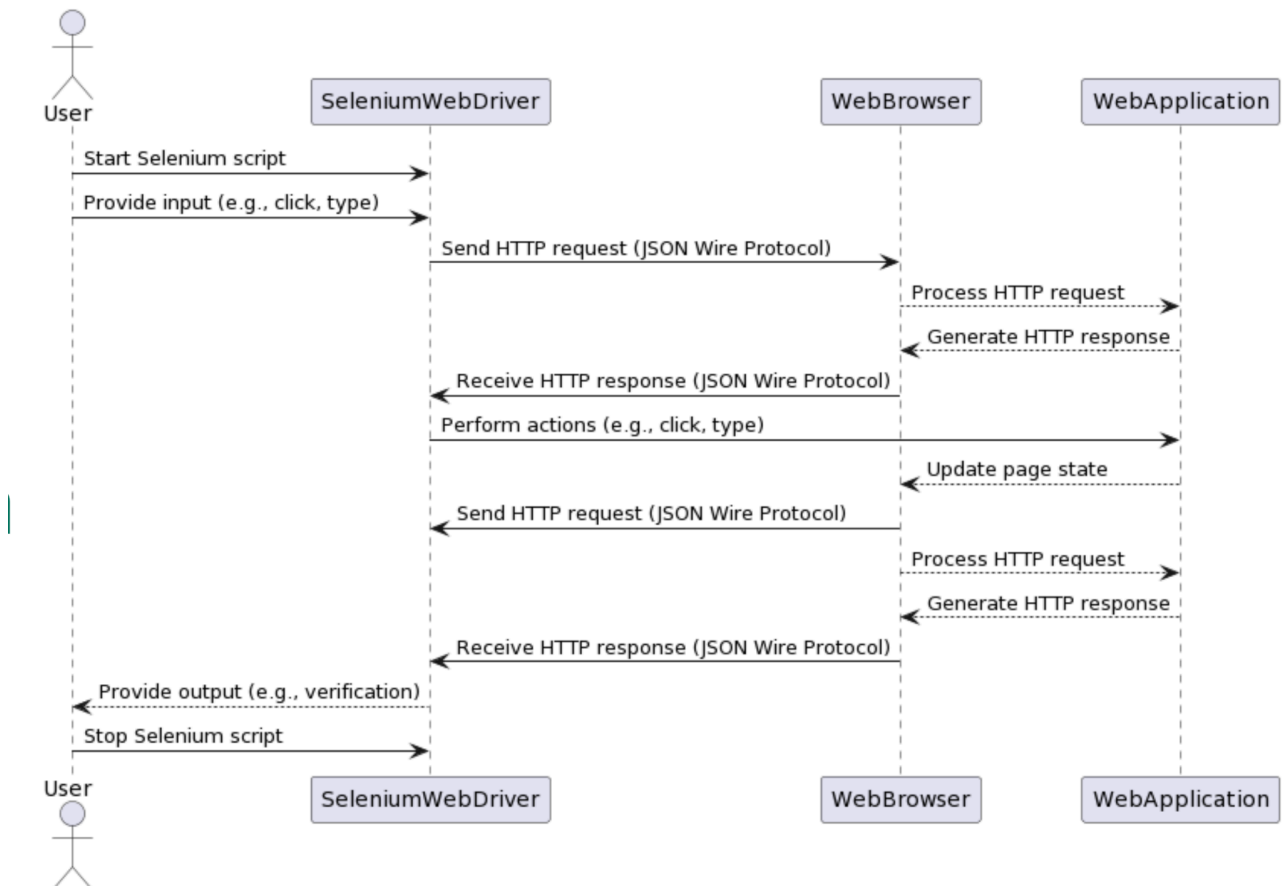


Fig-4.3.1: Sequence Diagram

4.3.2 USE CASE DIAGRAM - ONLINE TICKET BOOKING TEST CASE

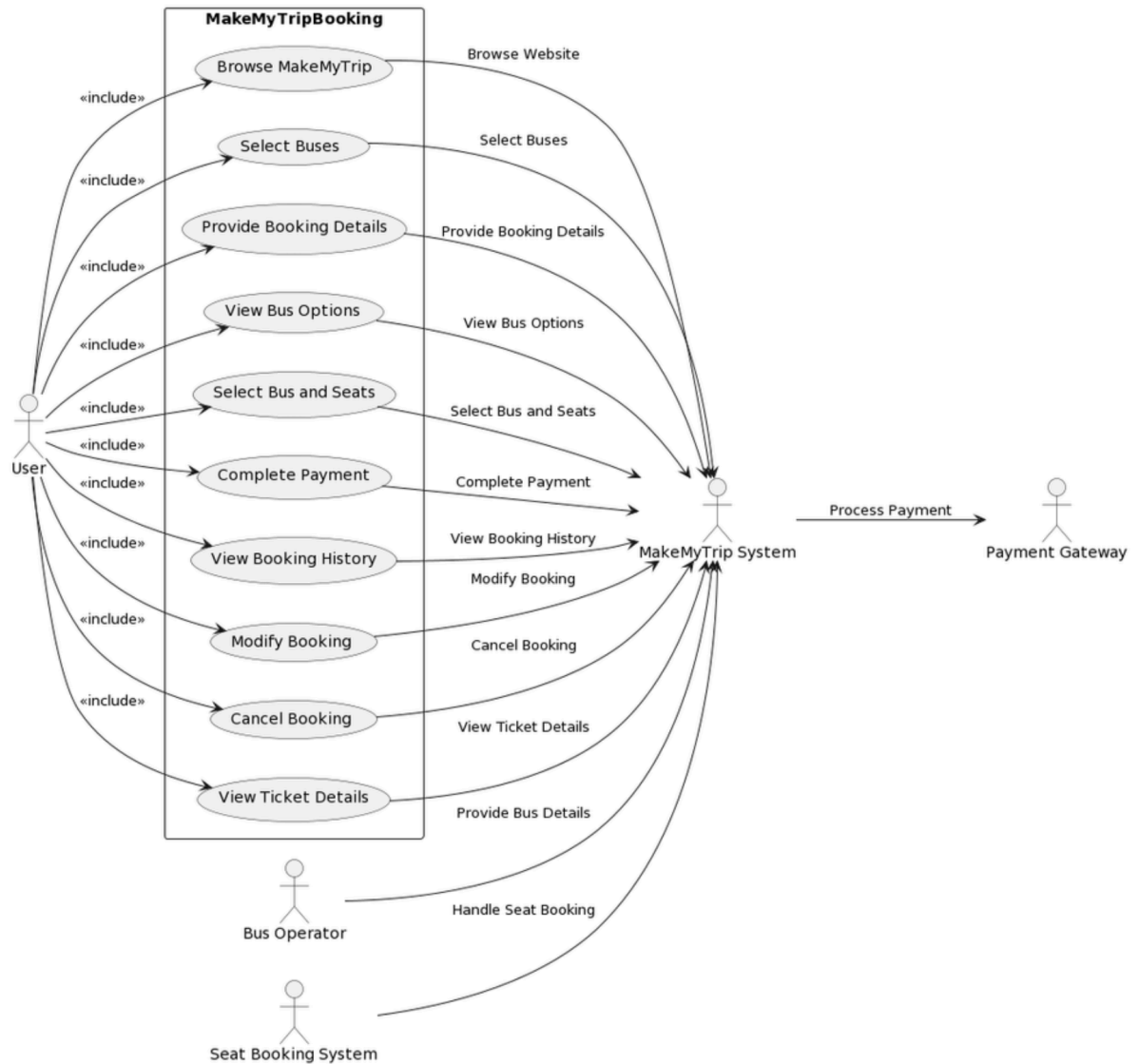


Fig-4.3.2: Use Case Diagram

4.3.3 CLASS DIAGRAM - AUTOMATION SYSTEM

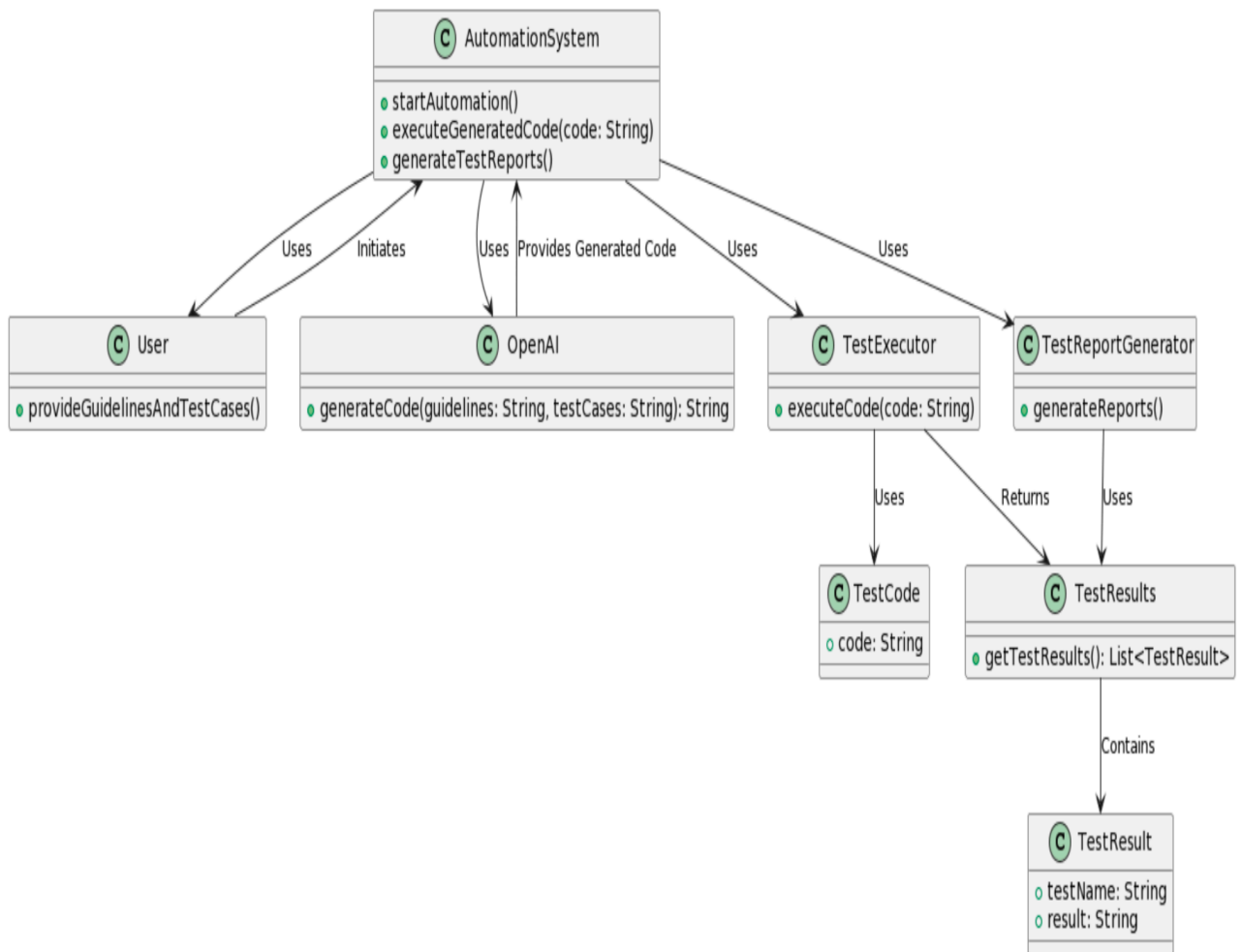


Fig-4.3.3: Class Diagram

IMPLEMENTATION AND RESULTS

5. IMPLEMENTATION AND RESULT

5.1 INTRODUCTION

The implementation phase of our project strategically integrates a suite of powerful testing tools to achieve our overarching objective of revolutionizing the automation testing process. Our toolset includes Selenium WebDriver for executing UI tests on diverse web applications, Rest Assured for API test execution, TestNG for test organization and parallel execution, Maven for project management, and Extent Reports for generating comprehensive test reports. A pivotal component of our implementation involves the seamless integration of OpenAI GPT, a cutting-edge language model, to introduce AI-driven automation. The flow entails sending manual test guidelines and cases to the OpenAI API, which processes this natural language input and returns automation scripts for UI and API tests. These scripts are then executed by the respective testing tools, with Selenium WebDriver mimicking user interactions and Rest Assured validating API responses. TestNG orchestrates efficient test execution, and Maven ensures streamlined project management. The tool generates detailed reports using Extent Reports, highlighting the success or failure of each test case.

5.2 SOURCE CODE

Java Application UI Code:

```
package org.example;

import javax.swing.*;
import java.awt.*;
import java.io.*;
import java.net.URI;
import java.util.Arrays;
import org.testng.TestNG;

public class GenerateUI {

    static JTextArea logArea;

    private static final String basePath = "src/main/java/org/example/";

    public static void main(String[] args) {

        SwingUtilities.invokeLater(() -> createAndShowGUI());

    }

    private static void createAndShowGUI() {

        JFrame frame = new JFrame("AutoDriver - UI");

        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        frame.setSize(1600, 800);

        // Input components

        JTextArea textArea = new JTextArea(15, 40);

        textArea.setLineWrap(true);

        textArea.setWrapStyleWord(true);

        JButton runUAButton = new JButton("Generate and Run UI Automation Scripts");
```



```
// Output components

logArea = new JTextArea(25, 40);

logArea.setEditable(false);

JScrollPane logScrollPane = new JScrollPane(logArea);


// Layout

JPanel inputPanel = new JPanel();

inputPanel.add(new JLabel("Enter Manual UI Test Cases:"));

inputPanel.add(new JScrollPane(textArea));

inputPanel.add(runUAButton);

JPanel outputPanel = new JPanel();

outputPanel.add(new JLabel("Logs:"));

outputPanel.add(logScrollPane);

Container contentPane = frame.getContentPane();

contentPane.setLayout(new BorderLayout());

contentPane.add(inputPanel, BorderLayout.NORTH);

contentPane.add(outputPanel, BorderLayout.CENTER);


runUAButton.addActionListener(e -> {

// Execute the Button ActionListener content here

try {

    String fileName = basePath + "uitestcases.txt";

    String inputText = textArea.getText();

    File file = new File(fileName);

    if (file.exists()) {

        file.delete();
```

```
    }

    BufferedWriter bw = new BufferedWriter(new FileWriter(fileName, true));

    bw.write(inputText);

    bw.close();

    logArea.append("Text replacement successful!\n");
} catch (IOException ex) {

    logArea.append("An error occurred: " + ex.getMessage() + "\n");

}

// Run the TestNG suite

runTestNGSuite(basePath + "generateandrunui.xml", "generateandrunui.xml");

});

frame.setVisible(true);

}

private static void runTestNGSuite(String suiteFilePath, String suiteName) {

    logArea.append("Running TestNG suite: " + suiteName + "... \n");

    try {

        TestNG testng = new TestNG();

        testng.setTestSuites(Arrays.asList(new String[] {suiteFilePath}));

        // Capture the standard output and error streams

        ByteArrayOutputStream outputStream = new ByteArrayOutputStream();

        ByteArrayOutputStream errorStream = new ByteArrayOutputStream();

        PrintStream outStream = new PrintStream(outputStream);

        PrintStream errStream = new PrintStream(errorStream);
```

```
System.setOut(outStream);

System.setErr(errStream);

testng.run();

// Display logs from the standard output and error streams

logArea.append(outputStream.toString());

logArea.append(errorStream.toString());

logArea.append("TestNG suite " + suiteName + " executed successfully.\n");

} catch (Exception ex) {

    logArea.append("An error occurred while running TestNG: " + ex.getMessage() +
"\n");

}

}

private static void openTestNGReportInBrowser() {

    try {

        File reportFile = new File(basePath + "test-output/dashboard.html");

        if (reportFile.exists() && Desktop.isDesktopSupported()) {

            String reportPath = reportFile.getAbsolutePath().replace("\\", "/");

            Desktop.getDesktop().browse(new URI("file:/// " + reportPath));

        } else {

            logArea.append("Report file not found or unable to open in a browser.\n");

        }

    } catch (Exception ex) {

        logArea.append("An error occurred while opening the report: " + ex.getMessage() +
"\n");

    }

}
```

OpenAI GPT API Code:

```
package org.example;

import io.restassured.RestAssured;

import io.restassured.response.Response;

import org.json.simple.parser.JSONParser;

import org.json.simple.parser.ParseException;

import org.testng.annotations.Test;

import java.io.File;

import java.io.FileWriter;

import java.io.IOException;

import static io.restassured.RestAssured.given;

public class UIchatgpt extends ReadJsonFile {

    @Test

    public void codeGeneration() throws IOException, ParseException {

        String guidelines = "src/main/java/org/example/guidelinesUI.txt";

        String testCases = "src/main/java/org/example/uitestcases.txt";

        JSONParser parser = new JSONParser();

        String guide = readJsonFile(guidelines);

        String testCase = readJsonFile(testCases);

        RestAssured.baseURI = "https://api.openai.com/v1/chat/completions";

        String token = "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";

        System.out.println("TestCases has been passed to ChatGPT\n Code generation in progress
        ...");

        String requestBody = "{\n" +
```

```
"  \"model\": \"gpt-3.5-turbo\", \"n\" +
    \"  \"messages\": [\n\" +
    \"    {\n\" +
    \"      \"role\": \"user\", \"n\" +
    \"      \"content\": \"\" + guide + \" for following test cases \" + testCase + \"\"\"\\n\" +
    \"    }\n\" +
    \"  ],\n\" +
    \"  \"temperature\": 0.7\n\" +
    \"}\";
```

```
Response response = given()
    .header("Content-Type", "application/json")
    .header("Authorization", "Bearer " + token)
    .body(requestBody)
    .when()
    .post()
    .then()
    .statusCode(200)
    .extract()
    .response();

String content = response.jsonPath().get("choices[0].message.content");
String codeBlock = content;
if(content.contains("`")) {
    String a[] = content.split("`");
    codeBlock = a[1];
}
```

```
codeBlock.replaceAll("\\\\n", "\n");

codeBlock = codeBlock.replace("java\n", "");

System.out.println(codeBlock);

createClass(codeBlock);

}

public static String insertString(String codeBlock) {

    String originalString = "package org.example;\n";

    originalString = originalString + codeBlock;

    // return the modified String

    return originalString;

}

public void createJavaClass(String code) {

    String filename = "src/main/java/org/example/Sample.java";

    try {

        File file = new File(filename);

        FileWriter writer = new FileWriter(file);

        writer.write(code);

        writer.close();

        System.out.println("Java file created successfully.");

    } catch (IOException e) {

        System.out.println("An error occurred while creating the Java file.");

        e.printStackTrace();

    }

}
```

TestNG Xml Code:

```
<?xml version="1.0" encoding="UTF-8"?>

<!DOCTYPE suite SYSTEM "http://testng.org/testng-1.0.dtd">

<suite name="All Test Suite" parallel="tests" thread-count="1">

    <listeners>

        <listener class-name="org.example.ExtendTestNGITestListenerUI"/>

    </listeners>

    <test name="ui code execution and report generation" preserve-order="true">

        <classes>

            <class name="org.example.JavaApplicationUI">

            </class>

        </classes>

    </test>

</suite>
```

GPT Generated UI Test Cases Code:

```
package org.example;

import javax.swing.*;
import java.awt.*;
import java.io.*;
import java.util.Arrays;
import io.github.bonigarcia.wdm.WebDriverManager;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.support.ui.ExpectedConditions;
```

```
import org.openqa.selenium.support.ui.WebDriverWait;

import org.testng.Assert;

import org.testng.annotations.AfterClass;

import org.testng.annotations.BeforeClass;

import org.testng.annotations.Test;


public class UIAutomation {

    private WebDriver driver;


    @BeforeClass

    public void setup() {

        // Set up WebDriver

        WebDriverManager.chromedriver().setup();

        driver = new ChromeDriver();

        driver.manage().window().maximize();

    }


    @Test(priority = 1)

    public void testStandardUser() {

        openURL("https://www.saucedemo.com");

        enterCredentials("standard_user", "secret_sauce");

        clickOnLogin();

        openProduct("Sauce Labs Backpack");


        addToCart();

    }

}
```



```
clickOnCart();  
checkout();  
enterShippingDetails("Kaishu", "Thushu", "400001");  
clickOnContinue();  
clickOnFinish();  
assertThankYouMessageDisplayed();  
sleep(2000); // 2-second delay  
}
```

```
@Test(priority = 2)  
public void testProblemUser() {  
    openURL("https://www.saucedemo.com");  
    enterCredentials("problem_user", "secret_sauce");  
    clickOnLogin();  
    openProduct("Sauce Labs Backpack");  
    addToCart();  
    clickOnCart();  
    checkout();  
    enterShippingDetails("Kaishu", "Thushu", "400001");  
    clickOnContinue();  
    clickOnFinish();  
    assertThankYouMessageDisplayed();  
    sleep(2000); // 2-second delay  
}
```

```
private void openURL(String url) {
```

```
driver.get(url);  
sleep(2000);  
}
```

```
private void enterCredentials(String username, String password) {  
    WebElement usernameField = waitForElement(By.id("user-name"));  
    usernameField.sendKeys(username);  
  
    WebElement passwordField = waitForElement(By.id("password"));  
    passwordField.sendKeys(password);  
    sleep(2000);  
}
```

```
private void clickOnLogin() {  
    WebElement loginButton = waitForElement(By.id("login-button"));  
    loginButton.click();  
    sleep(2000);  
}
```

```
private void openProduct(String productName) {  
    WebElement productLink = waitForElement(By.linkText(productName));  
    productLink.click();  
    sleep(2000);  
}
```

```
private void addToCart() {
```

```
        WebElement addToCartButton = waitForElement(By.xpath("//button[text()='Add to  
cart']"));
        addToCartButton.click();
        sleep(2000);
    }
```

```
private void clickOnCart() {
    WebElement cartIcon = waitForElement(By.className("shopping_cart_link"));
    cartIcon.click();
    sleep(2000);
}
```

```
private void checkout() {
    WebElement checkoutButton = waitForElement(By.id("checkout"));
    checkoutButton.click();
    sleep(2000);
}
```

```
private void enterShippingDetails(String firstName, String lastName, String zip) {
    WebElement firstNameField = waitForElement(By.id("first-name"));
    firstNameField.sendKeys(firstName);
    WebElement lastNameField = waitForElement(By.id("last-name"));
    lastNameField.sendKeys(lastName);
    WebElement zipField = waitForElement(By.id("postal-code"));
    zipField.sendKeys(zip);
}
```

```
        sleep(2000);
    }

    private void clickOnContinue() {
        WebElement continueButton = waitForElement(By.id("continue"));
        continueButton.click();
        sleep(2000);
    }

    private void clickOnFinish() {
        WebElement finishButton = waitForElement(By.id("finish"));
        finishButton.click();
        sleep(2000);
    }

    private void assertThankYouMessageDisplayed() {
        WebElement thankYouMessage = waitForElement(By.xpath("//h2[text()='Thank you for your order!']"));
        Assert.assertTrue(thankYouMessage.isDisplayed(), "Thank you message is not displayed.");
        sleep(2000);
    }

    private WebElement waitForElement(By locator) {
        WebDriverWait wait = new WebDriverWait(driver, 10);
        return wait.until(ExpectedConditions.visibilityOfElementLocated(locator));
    }
```

```
private void sleep(long milliseconds) {  
    try {  
        Thread.sleep(milliseconds);  
    } catch (InterruptedException e) {  
        // Handle the exception if needed  
    }  
}
```

@AfterClass

```
public void teardown() {  
    driver.quit();  
}
```

UI Guidelines Example:

1. write test cases in java, selenium and testng
2. create methods for sendkeys and click with wait (sleep) of 2 secs
3. give package name as org.example
4. give class name as Uitest
5. Use bonigarcia Webdriver manager
6. Don't close the browser at the end
7. only ASSERT steps with following tag ASSERTION:

UI Manual Test Cases Example:

Steps

1. Open <https://www.saucedemo.com>
2. Enter username as standard_user, password as secret_sauce and click on login
3. Open the product Sauce Labs Backpack by clicking on it and then add to cart
5. Click on cart icon and checkout
6. Enter first name as Kaishu, last name as Thushu and zip as 400001 and click on continue button
7. Click on finish and complete the process
8. ASSERTION: Verify that exact message 'Thank you for your order!' is displayed
9. Also test above steps for account with following login details: username - problem_user and password - secret_sauce

5.3 OUTPUT CODE

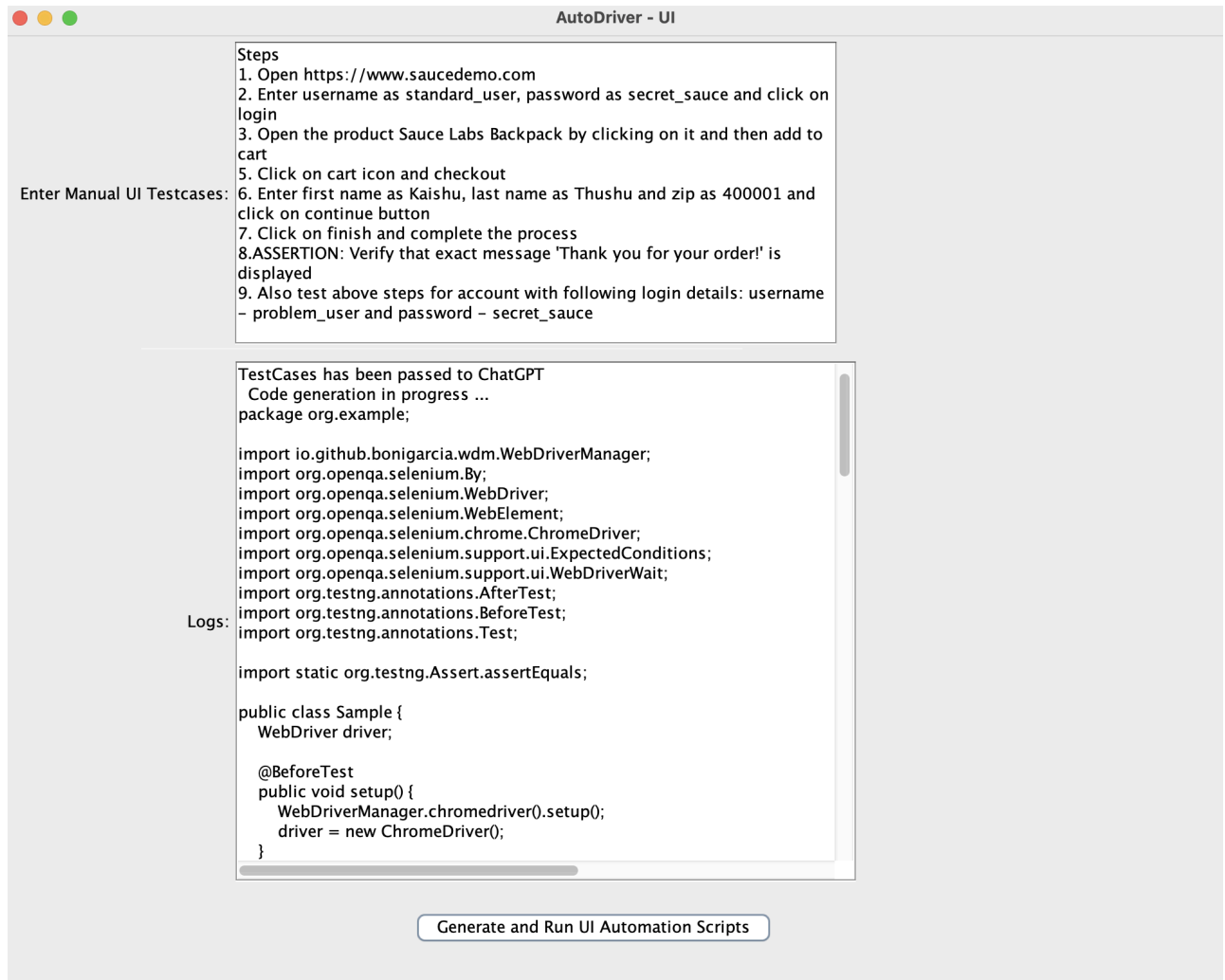


Fig-5.3.1: AI Automation Tool Interface

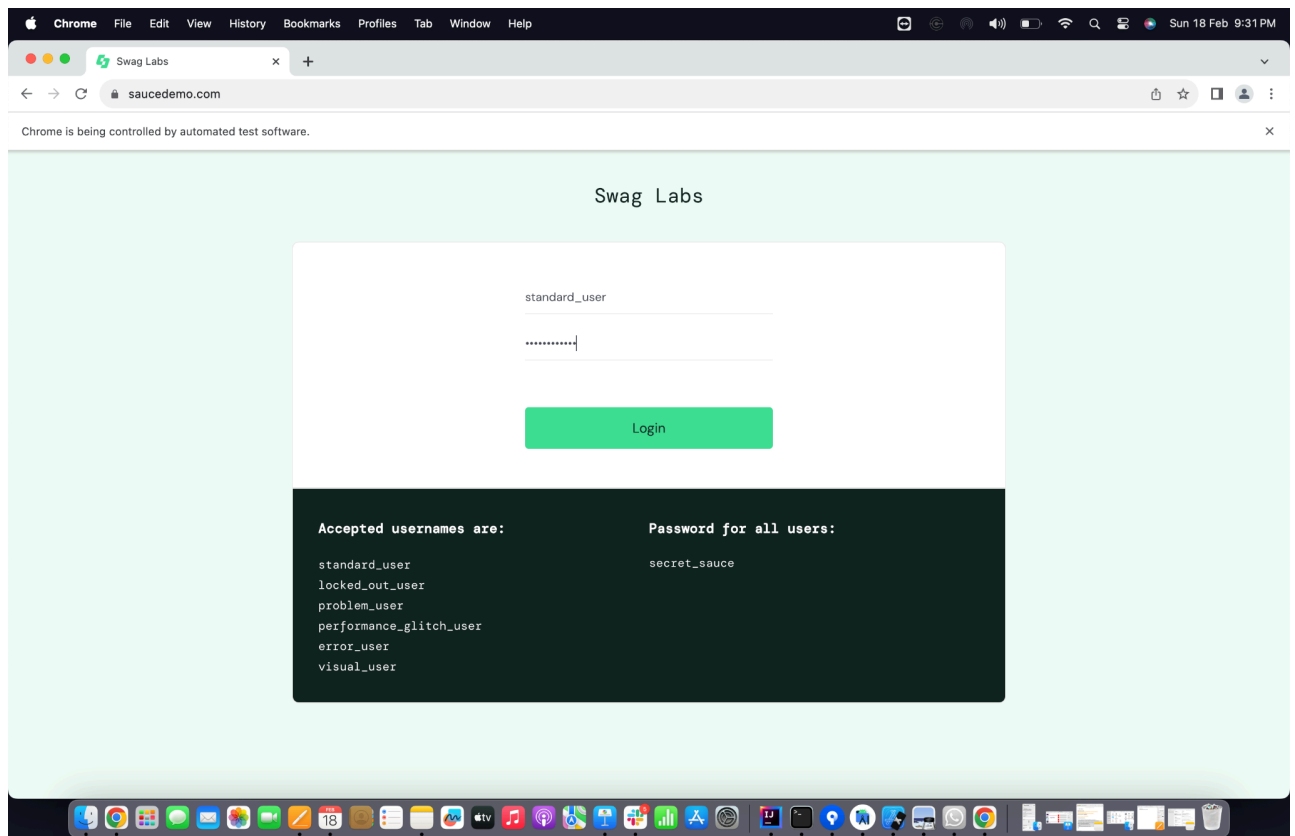


Fig-5.3.2: Automation Scripts Execution

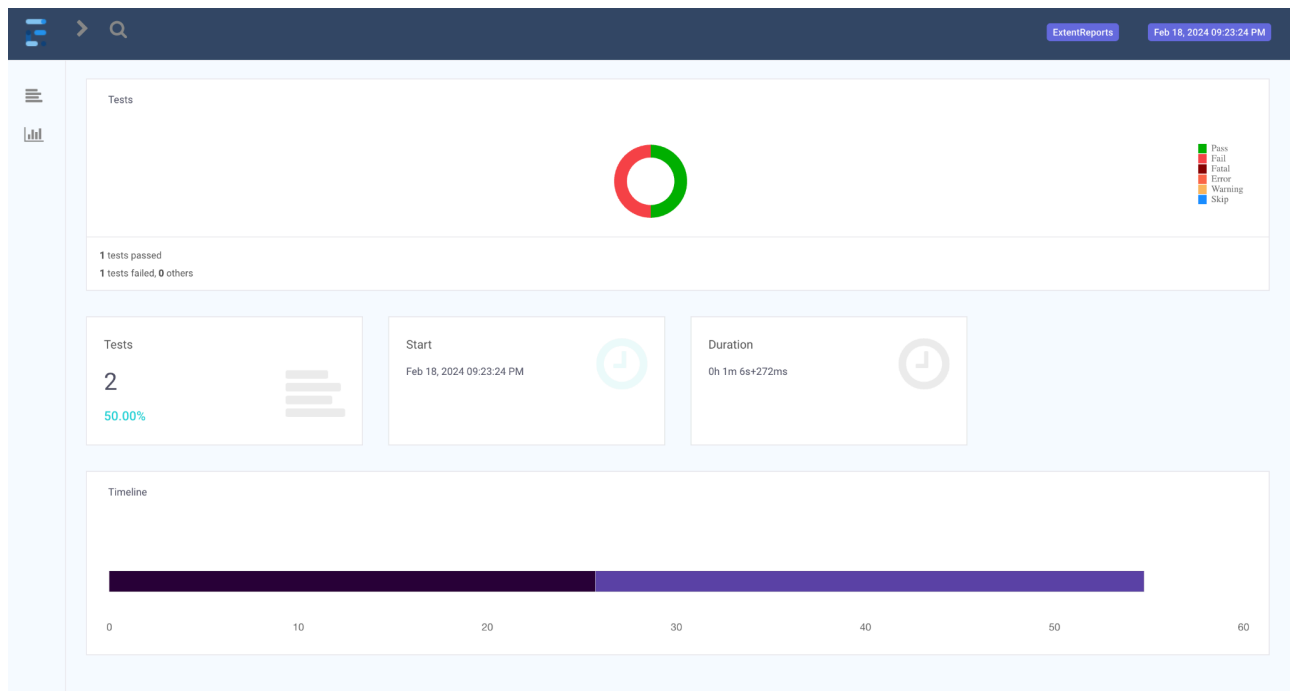


Fig-5.3.3: Reports Dashboard

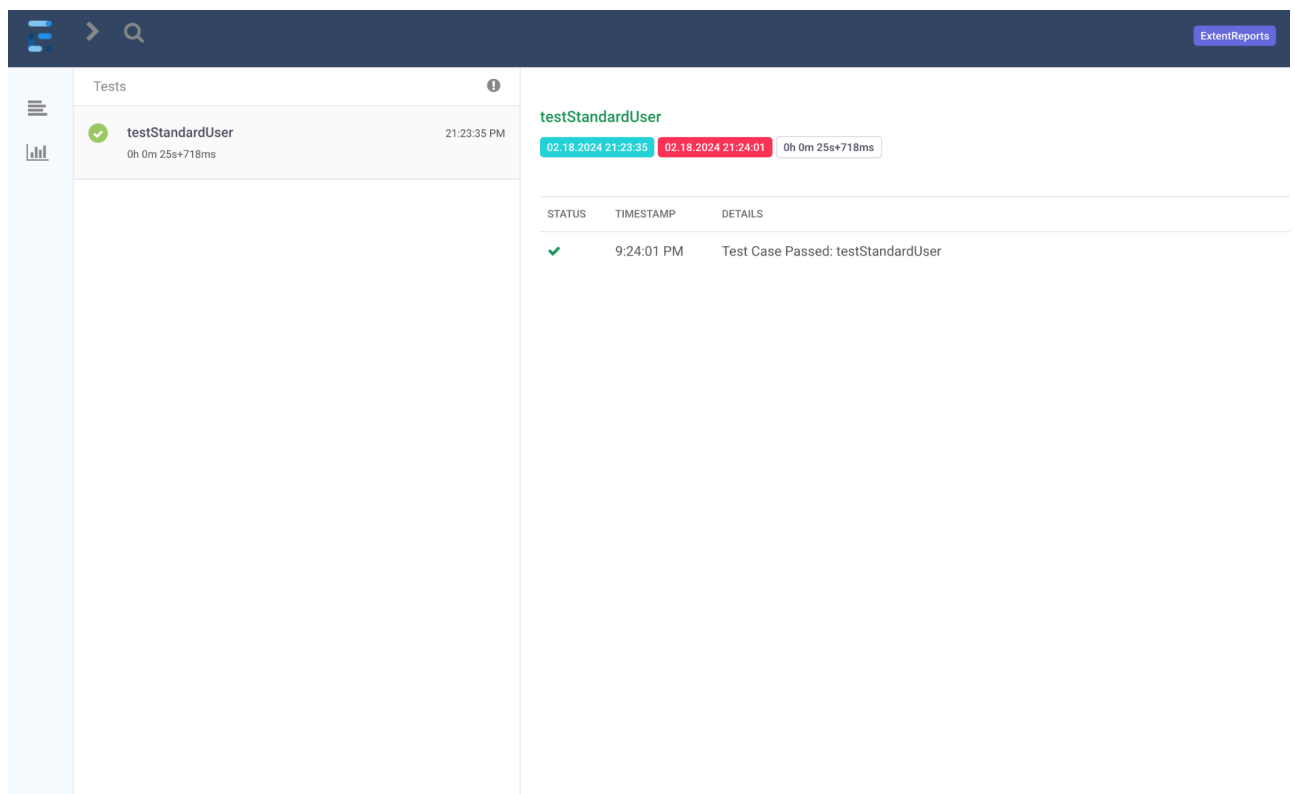


Fig-5.3.4: Passed Test Case

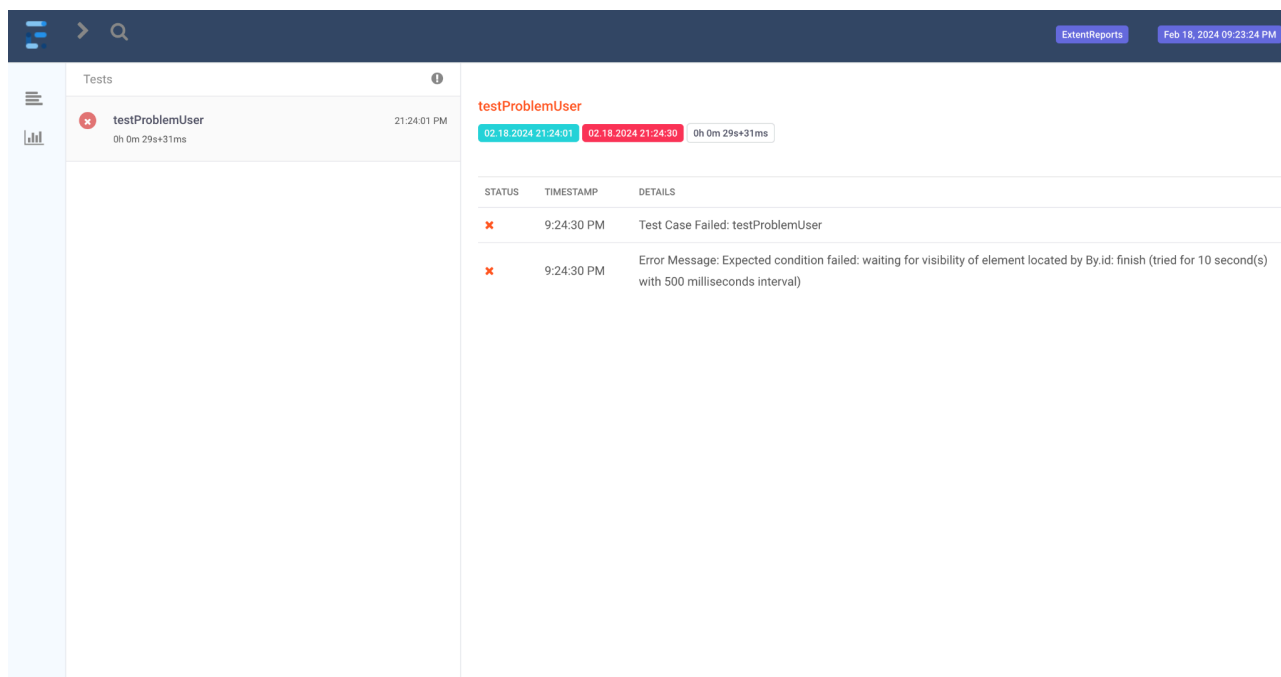


Fig-5.3.5: Failed Test Case

TESTING AND VALIDATION

6. TESTING AND VALIDATION

Automated Test Script Generation Testing:

- Test the AI's ability to understand and interpret manual test cases and guidelines correctly.
- Verify that the generated test scripts are accurate, executable, and align with predefined test cases.

Execution Workflow Testing:

- Confirm that the automated test scripts can be executed within the test environment without manual intervention.
- Ensure that the test execution workflow is efficient and can handle multiple test cases in sequence.

Test Execution Validation Testing:

- Validate that the execution of generated test scripts produces the expected outcomes.
- Verify that the AI can accurately log the results of test executions and provide actionable insights.

Error Handling and Logging Testing:

- Test the AI's capability to handle unexpected scenarios during test script generation and execution.
- Ensure that all events are logged accurately for future analysis and debugging.

User Interaction Testing:

- Assess the intuitiveness of initiating test case automation application.
- Ensure that users can easily enter manual test cases, run and open the generated test reports.

Security and Compliance Testing:

- Test the AI-generated code and the automation process for compliance with relevant security standards.
- Ensure that sensitive data used in test cases is handled and stored securely by the AI system.

Report Generation Validation:

- Test the completeness and accuracy of the AI-generated test reports against predefined benchmarks and manual test cases.
- Confirm that the test reports generated by the AI include all necessary information such as test execution steps, results, error logs, and debugging information as per the manual test cases.
- Ensure that the generated reports are clear, understandable, and actionable for all stakeholders involved in the testing process.

Feedback Loop Testing:

- Verify that the system supports a feedback loop where the AI learns from previous test executions to improve future test script generation.
- Test the system's ability to adapt to changes in test cases and guidelines based on user feedback.

CONCLUSION AND FUTURE ENHANCEMENTS

7. CONCLUSION AND FUTURE ENHANCEMENTS

7.1 CONCLUSION

Our tool can be a valuable asset for individuals without coding skills who aim to automate the testing process for their apps or websites. By supplying manual test cases to the OpenAI Model, the tool automatically generates the required code and executes tests, subsequently producing detailed reports that distinguish between successful and failed cases. This approach not only reduces human intervention but also significantly cuts down on time consumption, making testing more efficient and accurate. This tool contributes to more streamlined, error-free, and cost-effective testing processes.

7.2 FUTURE ENHANCEMENTS

In our continuous pursuit of refining our automation framework, we are strategically advancing its capabilities to incorporate two critical dimensions: load testing and visual testing. This evolutionary step aims to empower users with a comprehensive testing suite that not only ensures optimal performance under regular conditions but also rigorously evaluates system behavior under varying loads and visually inspects application interfaces for any anomalies. Additionally, within our tool, we have recognized occasional false failures arising from artificial hallucinations. To enhance the tool's reliability, we are actively working on minimizing these instances of false failures. This ongoing improvement initiative aims to deliver a more comprehensive and reliable solution for users across various testing domains, addressing both their testing requirements and concerns regarding false results.

REFERENCES

8. REFERENCES

- [1] Analysis and Design of Selenium WebDriver Automation Testing Framework
<https://doi.org/10.1016/j.procs.2015.04.038>
- [2] The Impact of Artificial Intelligence on Software Testing
<https://ieeexplore.ieee.org/document/8717439>
- [3] Automated Web Testing with Machine Learning and Containers
<https://ieeexplore.ieee.org/document/10017868/references#references>
- [4] Scriptless GUI Automation Testing Tool for Web Applications
<https://www.sciencegate.app/document/10.35940/ijrte.a5849.0510121>
- [5] Automation Testing Tools: A Comparative View
https://www.researchgate.net/publication/346109409_AUTOMATION_TESTING_TOOL_S_A_COMPARATIVE_VIEW